

MDTS4313_737_4

SULAGNA ROY

2026-08-16

Problem Set 4 : Kernel Smoother

1. Write the following lines of code in R.

```
set . seed (1234)
```

```
X=runif(100 ,5 ,15)
```

```
Y=5sin(X)+23cos(X)^2+rnorm(100 ,0 ,5)
```

```
Sim.1=data.frame(X=X,Y=Y)
```

```
reg=function(x){5sin(x)+23cos(x)^2}
```

The function “reg” represents the truth (i.e., the regression function) of the simulated dataset.

Seed for Reproducible Outputs

```
set.seed (1234)
```

100 obs.s on predictor X from U(5,15)

```
X=runif(100,5,15)
```

100 obs. on response Y (errors~N(mean=0,sd=5))

```
Y=(5*sin(X))+(23*cos(X)^2)+rnorm(100,0,5)
```

Full Data Set

```
Sim.1=data.frame(X=X,Y=Y)
```

100 obs. on the true regression function.

```
reg=function(x){5*sin(x)+23*cos(x)^2}
```

2. Divide the X values into parts. Call the first 80 values the training test. Denote them by X.train. The rest of the 20 values constitute the test set. Call them X.test.

Training Set of Predictor X

```
X.train=X[1:80]
```

Test Set of Predictor X

```
X.test=X[81:100]
```

3. Divide the Y values in training set (Y.train) and test set (Y.test) in the similar way.

Training Set of Response Y

```
Y.train=Y[1:80]
```

Test Set of Response Y

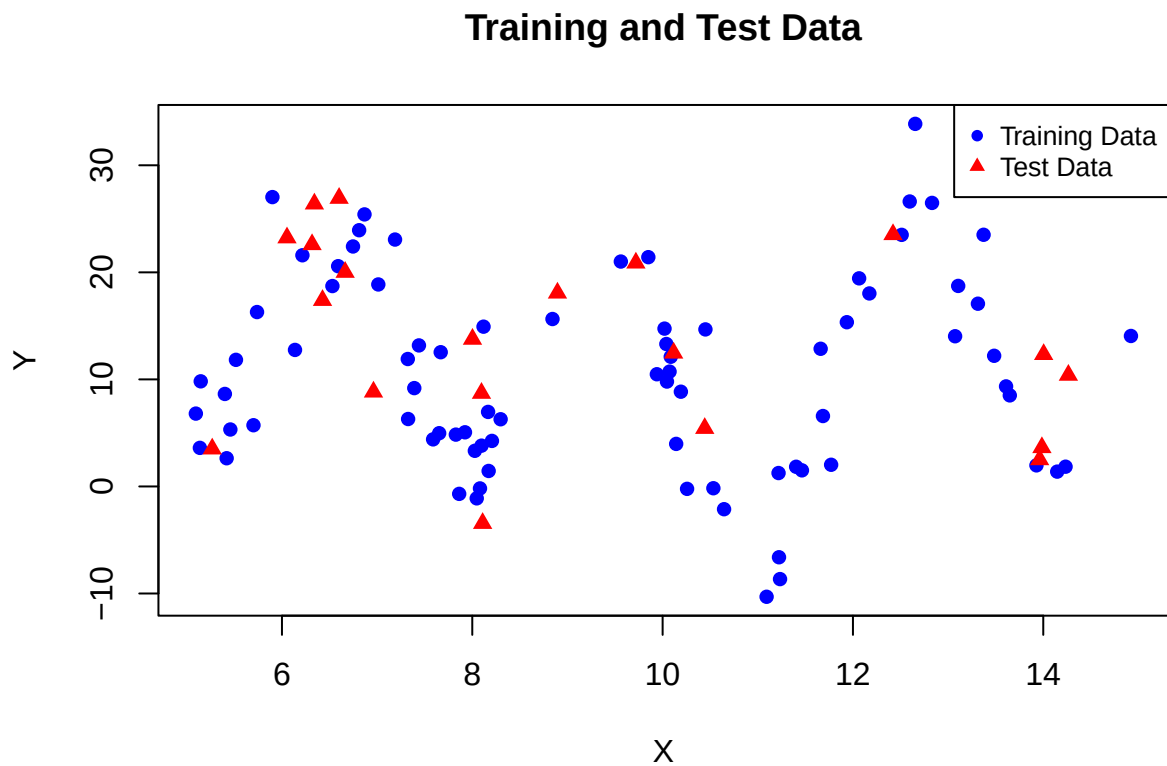
```
Y.test=Y[81:100]
```

4. Plot the data such that the points corresponding to the training set and the test set are represented with different symbols.

```
# Filled circle for training
plot(X.train,Y.train,pch = 16,col="blue",
     xlim = range(X),ylim=range(Y),
     xlab="X",ylab="Y",main="Training and Test Data")

# Add test observations(# Triangle for test )
points(X.test,Y.test,pch=17,col="red")

# Add legend
legend("topright",legend=c("Training Data","Test Data"),pch = c(16, 17),col = c("blue", "red"),cex=0.8)
```



5. Plot the regression function in the same graph.

```
# Create a sequence of X values for a smooth regression curve
x.grid=seq(min(X),max(X),length.out=500)

# Calculate the true regression function
y.true=reg(x.grid)

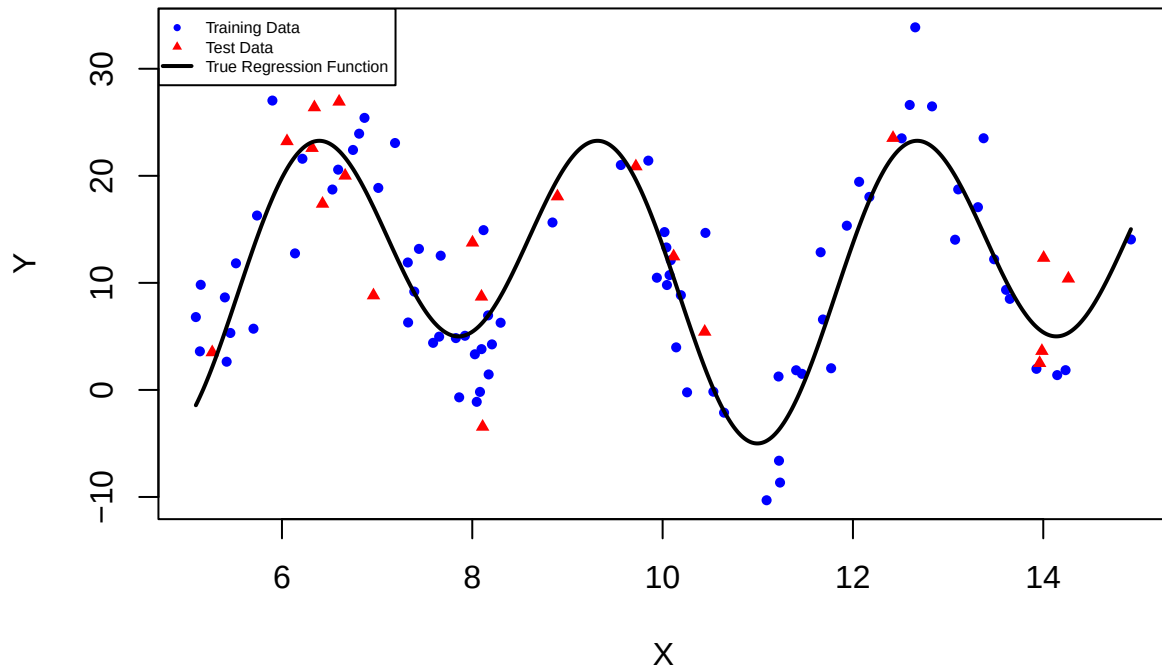
# Filled circle for training
plot(X.train,Y.train,pch = 16,col="blue",cex=0.7,
     xlim = range(X),ylim=range(Y),
     xlab="X",ylab="Y",main="Training and Test Data")

# Add test observations(# Triangle for test )
points(X.test,Y.test,pch=17,col="red",cex=0.7)

# Add true regression function
lines(x.grid,y.true,col="black",lwd = 2)

# Add legend
legend("topleft",
      legend=c("Training Data","Test Data","True Regression Function"),
      pch=c(16,17,NA),
      lty=c(NA,NA,1),
      col=c("blue","red","black"),
      lwd=c(NA,NA,2),
      cex=0.5)
```

Training and Test Data



6. Apply the Kernel smoother algorithm on the training dataset with the following kernels

- (a) Gaussian with $\sigma = 0.1, 0.25, 0.5, 1, 2$.
- (b) Uniform Kernel with $h = 0.1, 0.25, 0.5, 1, 2$.
- (c) Triangular Kernel with $h = 0.1, 0.25, 0.5, 1, 2$.
- (d) Epanechnikov Kernel with $h = 0.1, 0.25, 0.5, 1, 2$.

For each case, plot the estimates in separate graphs. Comment on your findings (i.e., how do the estimates change with changing $h(\sigma)$?)

6.1 — Define Kernel Smoother

```
# Kernel Smoother Function

kernel.smooth <- function(x, y, x0, h, kernel) {

  u <- (x0 - x) / h

  if(kernel == "Gaussian")
    w <- exp(-u^2 / 2)

  if(kernel == "Uniform")
    w <- ifelse(abs(u) <= 1, 1, 0)

  if(kernel == "Triangular")
    w <- ifelse(abs(u) <= 1, 1 - abs(u), 0)

  if(kernel == "Epanechnikov")
    w <- ifelse(abs(u) <= 1, 1 - u^2, 0)

  sum(w * y) / sum(w)
}
```

6.2 — Set bandwidths and prediction grid

```
h.val <- c(0.1, 0.25, 0.5, 1, 2)

x.grid <- seq(min(X.train), max(X.train), length.out = 200)
```

6.3 — Gaussian Kernel

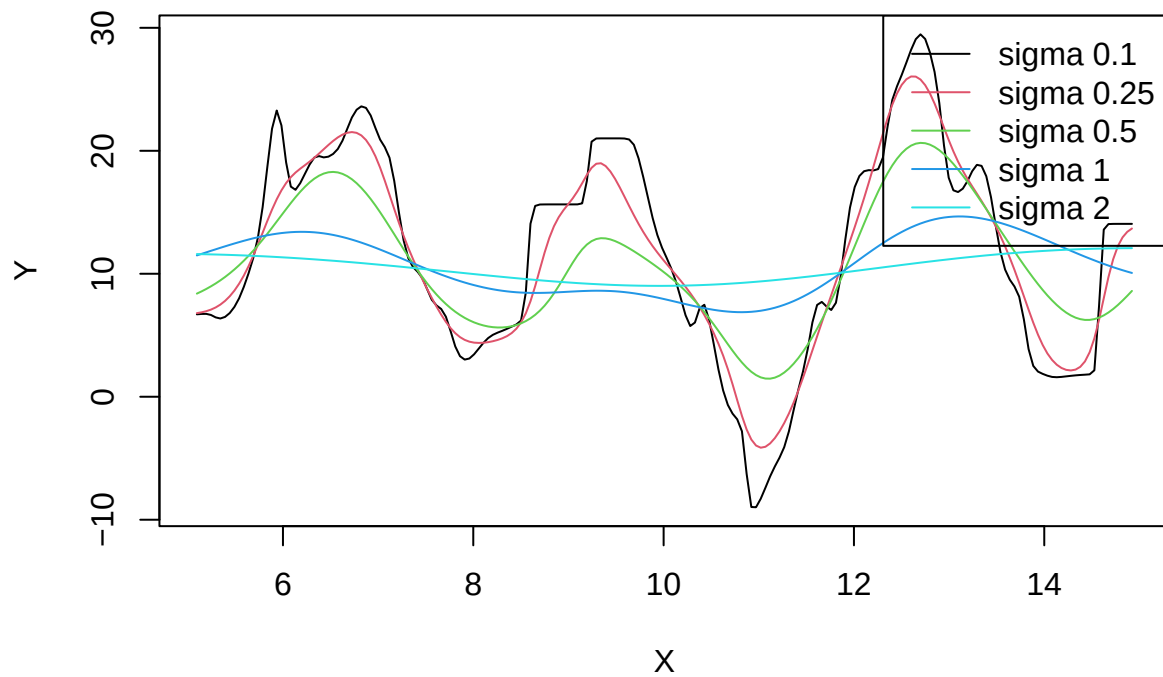
```
gauss.est <- matrix(NA, nrow = length(x.grid), ncol = 5)

for(j in 1:5)
  gauss.est[,j] <- sapply(x.grid, function(x)
    kernel.smooth(X.train, Y.train, x, h.val[j], "Gaussian"))

matplot(x.grid, gauss.est, type = "l", lty = 1,
        xlab = "X", ylab = "Y",
        main = "Gaussian Kernel")

legend("topright", legend = paste(expression(sigma), h.val),
       lty = 1, col = 1:5)
```

Gaussian Kernel



6.4 — Uniform Kernel

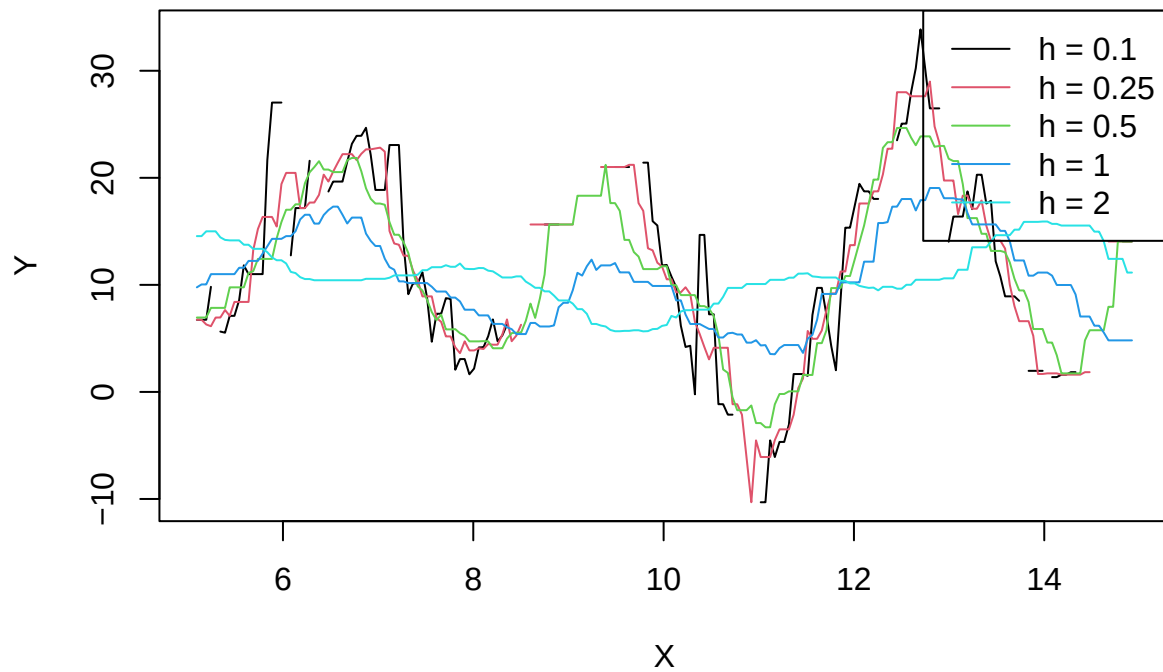
```
uniform.est <- matrix(NA, nrow = length(x.grid), ncol = 5)

for(j in 1:5)
  uniform.est[,j] <- sapply(x.grid, function(x)
    kernel.smooth(X.train, Y.train, x, h.val[j], "Uniform"))

matplot(x.grid, uniform.est, type = "l", lty = 1,
  xlab = "X", ylab = "Y",
  main = "Uniform Kernel")

legend("topright", legend = paste("h =", h.val),
  lty = 1, col = 1:5)
```

Uniform Kernel



6.5 — Triangular Kernel

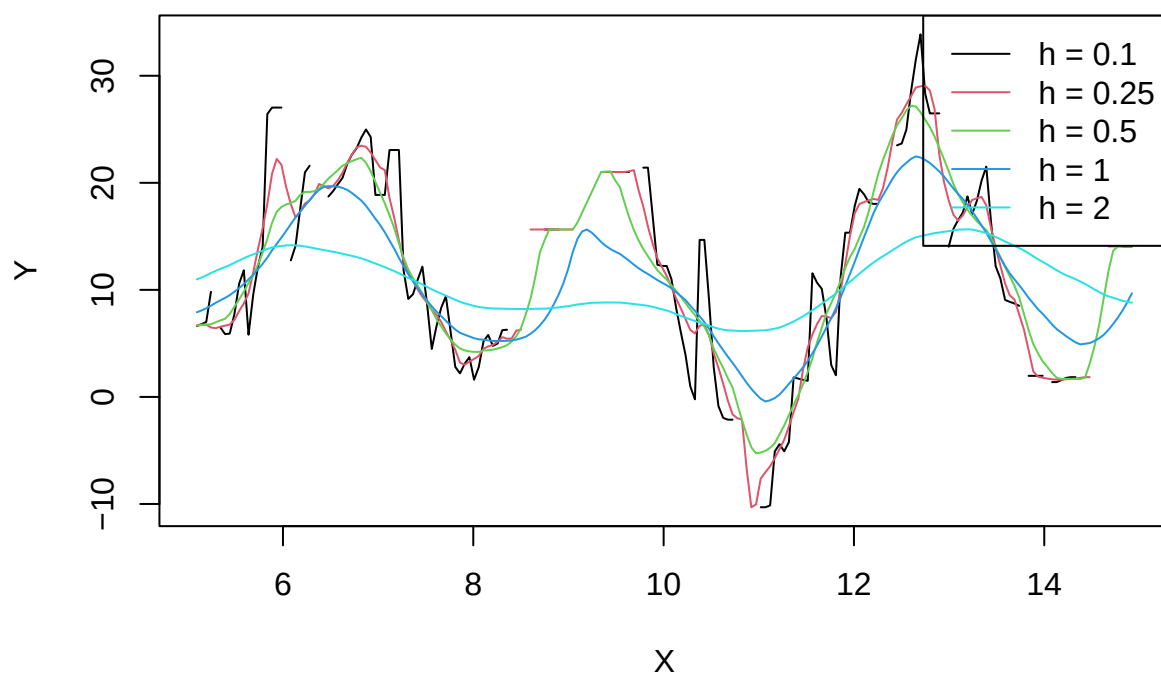
```
tri.est <- matrix(NA, nrow = length(x.grid), ncol = 5)

for(j in 1:5)
  tri.est[,j] <- sapply(x.grid, function(x)
    kernel.smooth(X.train, Y.train, x, h.val[j], "Triangular"))

matplot(x.grid, tri.est, type = "l", lty = 1,
  xlab = "X", ylab = "Y",
  main = "Triangular Kernel")

legend("topright", legend = paste("h =", h.val),
  lty = 1, col = 1:5)
```

Triangular Kernel



6.6 — Epanechnikov Kernel

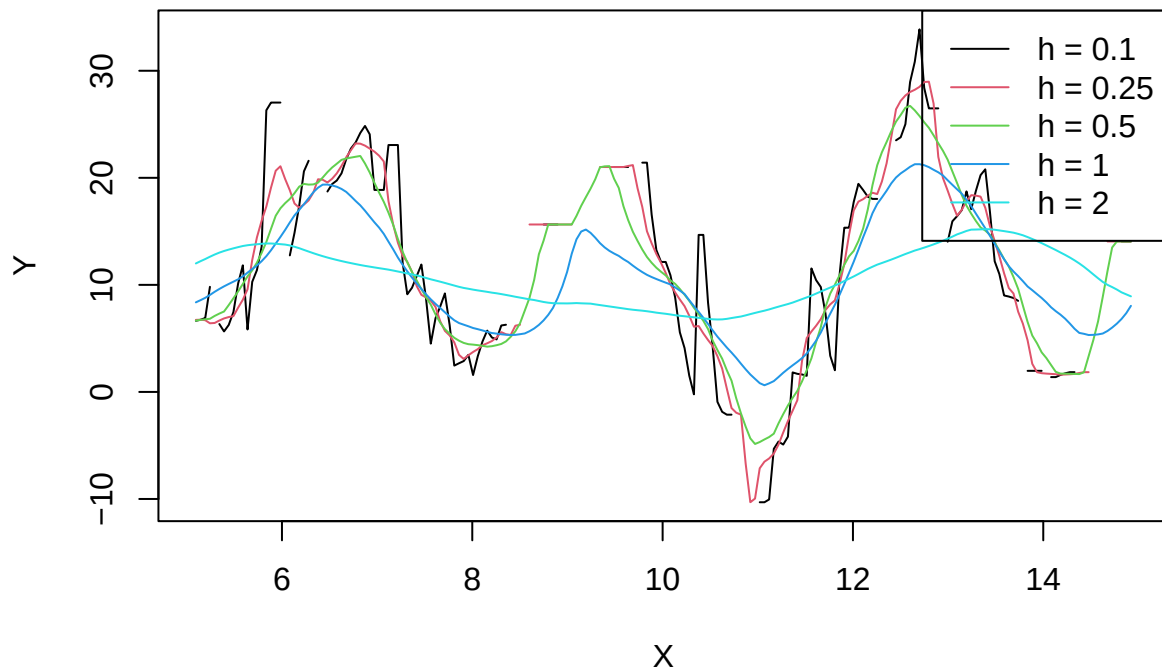
```
epa.est <- matrix(NA, nrow = length(x.grid), ncol = 5)

for(j in 1:5)
  epa.est[,j] <- sapply(x.grid, function(x)
    kernel.smooth(X.train, Y.train, x, h.val[j], "Epanechnikov"))

matplot(x.grid, epa.est, type = "l", lty = 1,
        xlab = "X", ylab = "Y",
        main = "Epanechnikov Kernel")

legend("topright", legend = paste("h =", h.val),
      lty = 1, col = 1:5)
```

Epanechnikov Kernel



Comment:

For smaller values of h (or σ), the kernel smoother follows the observations more closely and produces a more variable or wiggly estimate. As h increases, more observations receive substantial weight, resulting in a smoother curve. Very large bandwidths may oversmooth the data and fail to capture important features of the regression function.

7. For each case, compute the training and test errors.

7.1 — Error Function

```
MSE <- function(actual, predicted)
  mean((actual - predicted)^2)
```

7.2 — Calculate errors for all kernels

```

errors <- data.frame()

for(k in c("Gaussian", "Uniform", "Triangular", "Epanechnikov")) {

  for(h in h.val) {

    train.pred <- sapply(X.train, function(x)
      kernel.smooth(X.train, Y.train, x, h, k))

    test.pred <- sapply(X.test, function(x)
      kernel.smooth(X.train, Y.train, x, h, k))

    errors <- rbind(errors,
      data.frame(Kernel = k,
        h = h,
        Training.Error = MSE(Y.train, train.pred),
        Test.Error = MSE(Y.test, test.pred)))
  }
}

errors

```

##	Kernel	h	Training.Error	Test.Error
## 1	Gaussian	0.10	11.155573	35.81915
## 2	Gaussian	0.25	18.470190	31.91052
## 3	Gaussian	0.50	31.275718	38.19026
## 4	Gaussian	1.00	62.529862	72.35709
## 5	Gaussian	2.00	77.147803	85.30686
## 6	Uniform	0.10	10.120440	NaN
## 7	Uniform	0.25	19.331948	37.48875
## 8	Uniform	0.50	23.395220	30.89554
## 9	Uniform	1.00	42.921065	49.22875
## 10	Uniform	2.00	94.493427	111.72181
## 11	Triangular	0.10	5.055345	NaN
## 12	Triangular	0.25	11.595772	36.18093
## 13	Triangular	0.50	16.516854	32.77597
## 14	Triangular	1.00	26.266261	34.51852
## 15	Triangular	2.00	57.089164	67.48726
## 16	Epanechnikov	0.10	6.671673	NaN
## 17	Epanechnikov	0.25	14.483350	36.30778
## 18	Epanechnikov	0.50	18.485631	31.90394
## 19	Epanechnikov	1.00	30.406358	36.65361
## 20	Epanechnikov	2.00	68.836601	80.33977

7.3 — Find best bandwidth for each kernel

```

errors[order(errors$Test.Error), ]

##      Kernel    h Training.Error Test.Error
## 8      Uniform 0.50      23.395220  30.89554

```

## 18	Epanechnikov	0.50	18.485631	31.90394
## 2	Gaussian	0.25	18.470190	31.91052
## 13	Triangular	0.50	16.516854	32.77597
## 14	Triangular	1.00	26.266261	34.51852
## 1	Gaussian	0.10	11.155573	35.81915
## 12	Triangular	0.25	11.595772	36.18093
## 17	Epanechnikov	0.25	14.483350	36.30778
## 19	Epanechnikov	1.00	30.406358	36.65361
## 7	Uniform	0.25	19.331948	37.48875
## 3	Gaussian	0.50	31.275718	38.19026
## 9	Uniform	1.00	42.921065	49.22875
## 15	Triangular	2.00	57.089164	67.48726
## 4	Gaussian	1.00	62.529862	72.35709
## 20	Epanechnikov	2.00	68.836601	80.33977
## 5	Gaussian	2.00	77.147803	85.30686
## 10	Uniform	2.00	94.493427	111.72181
## 6	Uniform	0.10	10.120440	NaN
## 11	Triangular	0.10	5.055345	NaN
## 16	Epanechnikov	0.10	6.671673	NaN

7.4 - the best value for each kernel

```
do.call(rbind,
  lapply(split(errors, errors$Kernel),
    function(z) z[which.min(z$Test.Error), ]))
```

##		Kernel	h	Training.Error	Test.Error
##	Epanechnikov	Epanechnikov	0.50	18.48563	31.90394
##	Gaussian	Gaussian	0.25	18.47019	31.91052
##	Triangular	Triangular	0.50	16.51685	32.77597
##	Uniform	Uniform	0.50	23.39522	30.89554

Comment :

Smaller bandwidths give lower training errors but may overfit the data. Increasing the bandwidth produces smoother estimates and may reduce test error up to an optimal value. Very large bandwidths can cause underfitting and increase the test error. Thus, the bandwidth with minimum test error is preferred.

8. Use the `ksmooth` function in R to compare the results with your code for Gaussian Kernel only.

(Home Work: try to find any other inbuilt function in R to implement the other kernels.)

8.1 — Gaussian using `ksmooth()`

```
ks <- list()

for(h in h.val) {

  ks[[as.character(h)]] <-
    ksmooth(X.train, Y.train,
            kernel = "normal",
            bandwidth = h,
            x.points = x.grid)
}
```

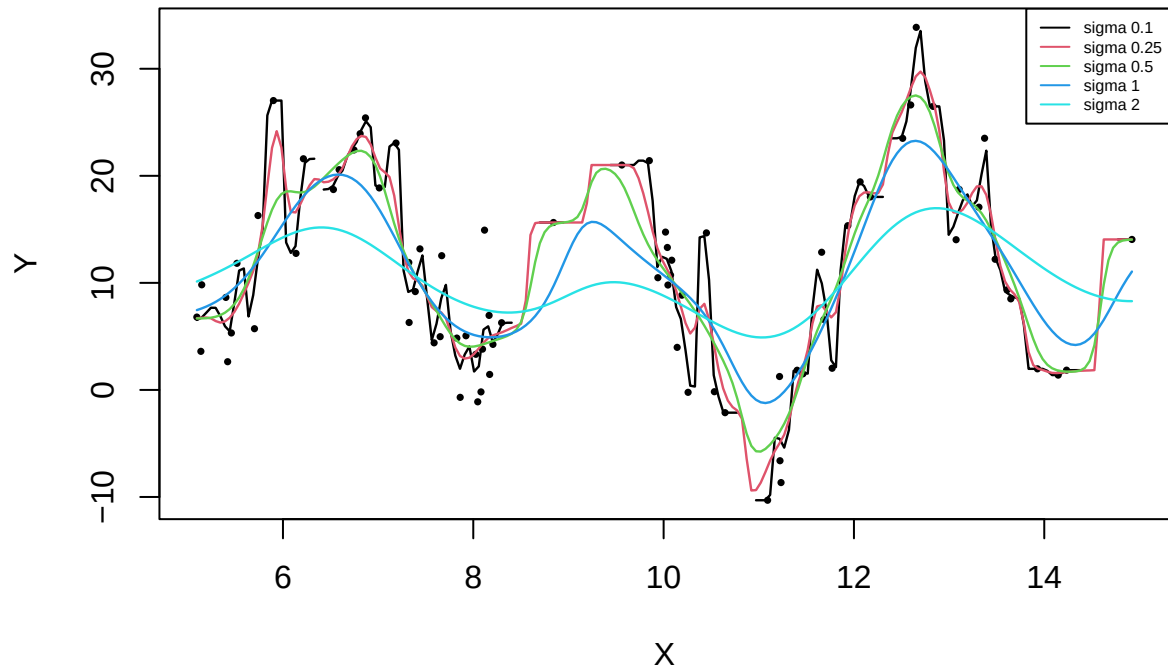
8.2 — Plot `ksmooth()` results

```
plot(X.train, Y.train,
     pch = 16, cex=0.5,
     xlab = "X", ylab = "Y",
     main = "Gaussian Kernel using ksmooth()")

for(j in 1:5)
  lines(ks[[j]]$x, ks[[j]]$y,
        lty = 1, lwd=1.2,
        col=j)

legend("topright",
      legend = paste(expression(sigma), h.val),
      lty = 1,
      col=1:5,
      cex=0.5)
```

Gaussian Kernel using ksmooth()



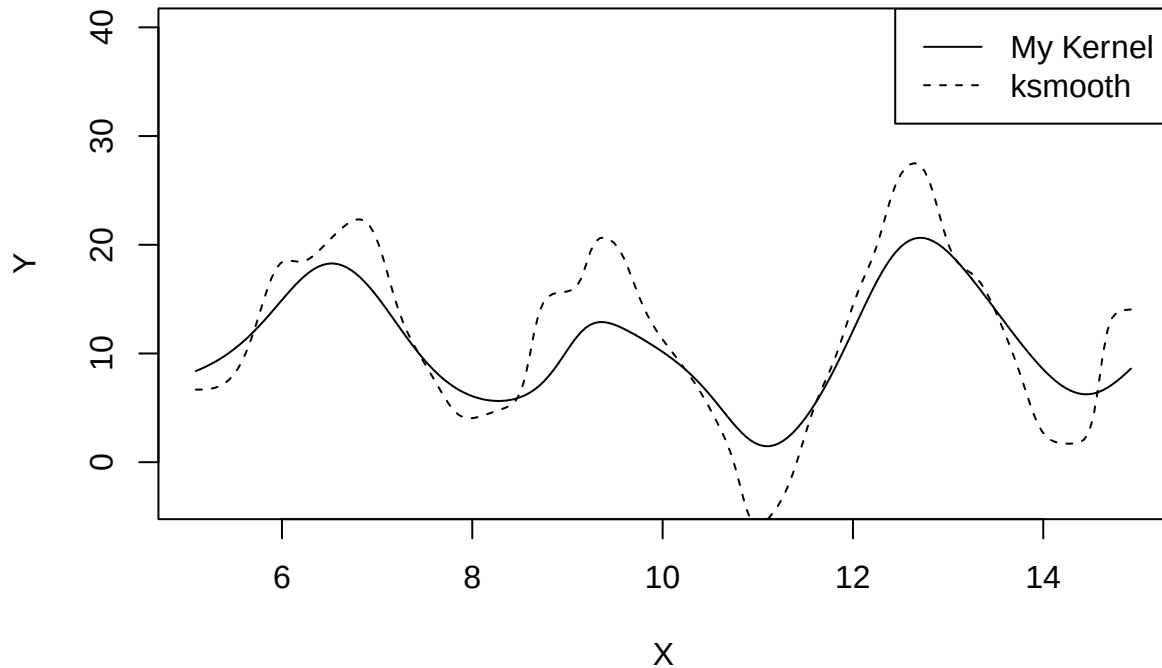
```
ks05 <- ksmooth(X.train, Y.train,
                 kernel = "normal",
                 bandwidth = 0.5,
                 x.points = x.grid)

plot(x.grid, gauss.est[,3],
     type = "l", ylim=c(-3.5,40),
     xlab = "X", ylab = "Y",
     main = "Comparison: Gaussian Kernel")

lines(ks05$x, ks05$y, lty = 2)

legend("topright",
      legend = c("My Kernel", "ksmooth"),
      lty = 1:2)
```

Comparison: Gaussian Kernel



Comment :

The Gaussian kernel smoother obtained using our own kernel-smoothing function is very similar to the result obtained using the built-in `ksmooth()` function in R. As the bandwidth $/sigma$ increases, both methods produce smoother curves, while smaller values of $/sigma$ give more flexible and wiggly estimates. Thus, `ksmooth()` provides results consistent with the manually implemented Gaussian kernel smoother. The question specifically asks for this comparison for the Gaussian kernel.

9. Repeat the procedure 50 times. Each time, the training set and the test set should be chosen randomly

from the given sample. Find the average of the training errors and

the testing errors thus obtained for each case and comment.

9.1 — 50 repetitions

```
set.seed(1234)
```

```

all.errors <- data.frame()

for(rep in 1:50) {

  ind <- sample(1:100, 80)

  x.tr <- X[ind]
  y.tr <- Y[ind]
  x.te <- X[-ind]
  y.te <- Y[-ind]

  for(k in c("Gaussian", "Uniform",
             "Triangular", "Epanechnikov")) {

    for(h in h.val) {

      train.pred <- sapply(x.tr, function(x)
        kernel.smooth(x.tr, y.tr, x, h, k))

      test.pred <- sapply(x.te, function(x)
        kernel.smooth(x.tr, y.tr, x, h, k))

      all.errors <- rbind(all.errors,
        data.frame(Rep = rep,
                    Kernel = k,
                    h = h,
                    Train.Error =
                      MSE(y.tr, train.pred),
                    Test.Error =
                      MSE(y.te, test.pred)))

    }
  }
}

```

9.2 — Average errors

```

avg.errors <- aggregate(cbind(Train.Error, Test.Error) ~ Kernel + h,
  data = all.errors,
  FUN = mean)

avg.errors

```

##	Kernel	h	Train.Error	Test.Error
## 1	Epanechnikov	0.10	7.704802	28.31160
## 2	Gaussian	0.10	12.429053	34.25241
## 3	Triangular	0.10	5.806184	28.54893
## 4	Uniform	0.10	11.867302	29.29858
## 5	Epanechnikov	0.25	15.399105	35.44484
## 6	Gaussian	0.25	18.894313	28.95739
## 7	Triangular	0.25	12.691749	35.35619

```
## 8      Uniform 0.25    20.157332    36.10895
## 9 Epanechnikov 0.50    19.000969    28.23313
## 10     Gaussian 0.50    30.075618    36.59500
## 11    Triangular 0.50    17.218456    28.79915
## 12      Uniform 0.50    22.883555    29.27768
## 13 Epanechnikov 1.00    29.099905    35.04801
## 14     Gaussian 1.00    62.397867    67.86782
## 15    Triangular 1.00    25.437011    32.84459
## 16      Uniform 1.00    41.287142    45.92094
## 17 Epanechnikov 2.00    69.265532    74.92746
## 18     Gaussian 2.00    77.812794    80.10076
## 19    Triangular 2.00    56.668793    63.18662
## 20      Uniform 2.00    97.344682   102.49904
```

9.3 — Find best bandwidth for each kernel

```
do.call(rbind,
        lapply(split(avg.errors, avg.errors$Kernel),
               function(z) z[which.min(z$Test.Error), ]))
```

```
##           Kernel      h Train.Error Test.Error
## Epanechnikov Epanechnikov 0.50    19.000969    28.23313
## Gaussian      Gaussian 0.25    18.894313    28.95739
## Triangular    Triangular 0.10     5.806184    28.54893
## Uniform       Uniform 0.50    22.883555    29.27768
```

Comment :

After repeating the random training-test split 50 times, the average errors provide a more reliable comparison of the kernels. Small bandwidths generally produce lower training error because the smoother follows the observations closely, but they may produce larger test error due to overfitting. Increasing the bandwidth smooths the estimates and can reduce test error up to an optimal bandwidth. Beyond this point, excessive smoothing can increase both bias and test error.

10. Consider the “Boston” dataset in the “MASS” library in R and let the variable “medv” be the response (Y) and the variable “lstat” be the predictor (X).

Use 22-fold cross-validation method to find the best choice of σ for applying the Gaussian Kernel smoother.

10.1 — Load Boston data

```
library(MASS)

data(Boston)

X.b <- Boston$lstat
Y.b <- Boston$medv
```

10.2 — 22-fold cross-validation

```
set.seed(1234)

K <- 22
fold <- sample(rep(1:K, length.out = nrow(Boston)))

sigma.val <- c(0.1, 0.25, 0.5, 1, 2)

cv.error <- numeric(length(sigma.val))

for(j in 1:length(sigma.val)) {

  err <- numeric(K)

  for(k in 1:K) {

    train <- which(fold != k)
    test <- which(fold == k)

    pred <- sapply(X.b[test], function(x)
      kernel.smooth(X.b[train],
                    Y.b[train],
                    x,
                    sigma.val[j],
                    "Gaussian"))

    err[k] <- mean((Y.b[test] - pred)^2)
  }

  cv.error[j] <- mean(err)
}

cv.result <- data.frame(sigma = sigma.val,
                       CV.Error = cv.error)

cv.result
```

```
##   sigma CV.Error
## 1  0.10 32.62593
## 2  0.25 29.47124
## 3  0.50 28.13449
## 4  1.00 28.07989
## 5  2.00 30.68215
```

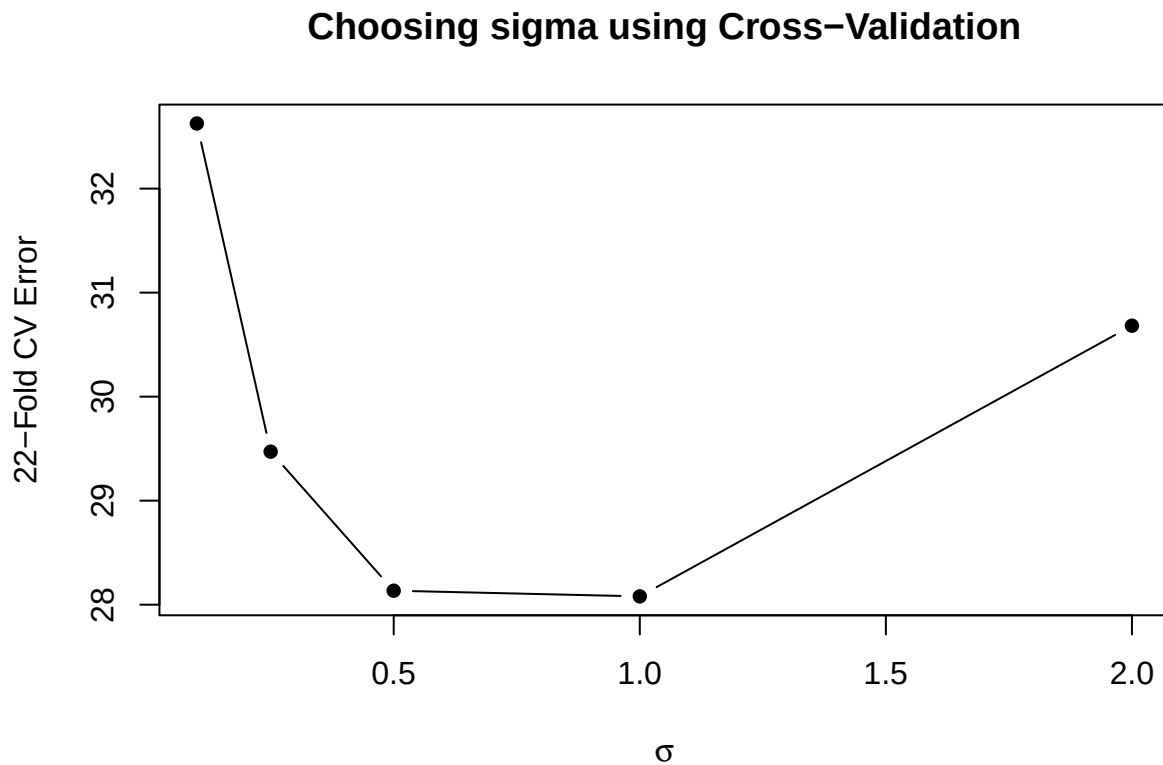
10.3 — Find best sigma

```
cv.result[which.min(cv.result$CV.Error), ]
```

```
##      sigma CV.Error  
## 4         1 28.07989
```

10.4 — Plot CV Error

```
plot(sigma.val, cv.error,  
      type = "b",  
      pch = 16,  
      xlab = expression(sigma),  
      ylab = "22-Fold CV Error",  
      main = "Choosing sigma using Cross-Validation")
```



```
cv.result[which.min(cv.result$CV.Error), ]
```

```
##      sigma CV.Error  
## 4         1 28.07989
```

Comment :

The optimal value of sigma is selected as the value corresponding to the minimum 22-fold cross-validation error. This value provides the best balance between under-smoothing and over-smoothing for predicting medv from lstat.
